

Dhrumil Parikh

AI Engineering Intern · Taazaa.inc · May – June 2026

WHAT I WORKED ON

1. Multimodal RAG Pipeline Evaluation — Free Stack vs Gemini

(DOCUMENTATION IN PROGRES)

Sales teams at Taazaa generate thousands of unsearchable files daily — recordings, contracts, business cards. Built two complete multimodal RAG pipelines from scratch and ran a head-to-head evaluation to give a grounded production recommendation.

Method 1 — Free Stack: Groq Whisper (transcription) + SpeechBrain ECAPA (speaker diarization) + Groq Vision llama-4-scout (OCR) + fastembed BAAI/bge 384-dim + BM25 + ChromaDB hybrid search + cross-encoder reranker + Groq Llama-3.3-70b. \$0 cost, 14 components.

Method 2 — Gemini Stack: All file types into Gemini File API (PDF, audio, video, images natively). Gemini Embedding-2 3072-dim + query expansion (4 rephrasings) + entity boosting + Gemini 2.5 Flash for reranking and generation. ~\$0.002/query, 2 AI components.

Evaluated on 12 questions via RAGAS (faithfulness, relevancy, precision, recall, correctness) run async through Celery, scores stored in PostgreSQL. Gemini won 4/5 — faithfulness 0.931 vs 0.757, context recall 0.931 vs 0.698. Recommendation: deploy Gemini ingestion with Method 1's BM25 + RRF + confidence gate for exact-phrase queries.

What I Will Add Next: Build the hybrid system. Fix **answer correctness** below target — likely a ground truth format mismatch, not a system failure and make a system which conveniently switches between both frameworks seamlessly.

2. Wandr — Multi-Agent AI Travel Planner ([DOCUMENTATION](#))

Built a multi-agent system on LangGraph StateGraph: 5 specialised agents (Destination, Weather, Budget, Itinerary, Packing) sharing an AgentState TypedDict. A deterministic Supervisor node (pure Python) validates each agent's output and injects specific feedback on failure before retrying. Live APIs: REST Countries, Open-Meteo, Open Exchange Rates. Deployed as FastAPI + SSE streaming + MemorySaver checkpointing. All 3 end-to-end test runs clean.

What I Will Add Next: Parallelize Destination, Weather, Budget agents via **LangGraph fan-out** — sequential execution wastes ~15s/trip.

3. LLM Fine-Tuning — Mistral-7B with QLoRA ([DOCUMENTATION](#))

Fine-tuned Mistral-7B on Dolly-15k using QLoRA (4-bit quantization + LoRA adapters on attention layers) — training only 0.36% of parameters on a single GPU. Base model rambles and hallucinates structure; fine-tuned model follows instructions cleanly. BLEU-2 +277%, ROUGE-2 +204%.

What I Will Add Next: Try **DPO** for preference alignment and **LoRA adapter merging** across domain-specific checkpoints.

4. Basic RAG — FAISS + PostgreSQL ([DOCUMENTATION](#))

Foundation project. Chunked a medical FAQ dataset (~16K docs, ~64K chunks), generated sentence embeddings via SentenceTransformers, indexed in FAISS, retrieved top-k chunks per query, passed context to an LLM, logged everything to PostgreSQL. Built the mental model that every later project extended.

WHAT I LEARNED

Benchmarking taught me intuition about AI tools is unreliable without measurement. I assumed the free stack would be clearly worse. It wasn't — it won on exact-phrase recall and speaker diarization. RAGAS gave a language for failure modes: low context recall means retrieval of missed content; low faithfulness means the LLM fabricated. Without metrics, both just look like "wrong answers."

Biggest architectural lesson: deterministic code beats LLMs for control logic. The Wandr Supervisor is pure Python — no LLM calls, instant debugging, auditable routing. Same with the confidence gate in the RAG pipelines — a simple cosine threshold that stops the LLM being called on bad retrieval.

Building two systems instead of one was the best decision. The head-to-head comparison revealed gaps neither system had alone — Method 1's BM25 + RRF handled exact-phrase queries Gemini's vector search missed. The real recommendation wasn't "pick Gemini" — it was "use Gemini's ingestion with Method 1's retrieval logic." That nuance only exists because both were built and measured.